

Report conversation

How to program a GitHub bot that reacts to `/format` messages on a PR by checking out the PR branch, running `prettier -w` and committing and pushing the changes?

To program a GitHub bot that reacts to `/format` messages on a Pull Request (PR) and performs the specified actions, you can follow these steps:

1. Set up a GitHub App:

- Go to the GitHub Developer settings and create a new GitHub App.
- Specify the necessary details for your app, such as the name, description, and homepage URL.
- Configure the app permissions to have access to the repositories where you want the bot to operate.
- Generate a private key for your GitHub App.

2. Create a new repository for your bot code:

- Initialize a new Git repository on your local machine.
- Create a new directory for your bot code and navigate to it.
- Initialize the directory as a new Git repository using `git init`.
- Create a new file, e.g., `bot.js`, and open it in your preferred code editor.

3. Install the required dependencies:

- Initialize a new Node.js project using `npm init`.
- Install the `probot` framework, which provides tools and utilities for building GitHub Apps:

```
nginx
```

 Copy code

```
npm install probot
```

4. Implement the bot logic:

- In the `bot.js` file, import the necessary modules and create a new Probot app:

```
javascript
```

 Copy code

```
const { Probot } = require('probot');  
const exec = require('child_process').exec;
```

```
const app = new Probot();
```

- Create an event listener that triggers when a comment is created on a PR:

javascript

 Copy code

```
app.on('issue_comment.created', async (context) => {
  const { comment, issue, repository } = context.payload;

  if (issue.pull_request && comment.body.trim() === '/format') {
    const { head } = issue.pull_request;
    const cloneUrl = repository.clone_url;
    const branchName = head.ref;

    // Clone the repository
    exec(`git clone ${cloneUrl}`, (err, stdout, stderr) => {
      if (err) {
        console.error(err);
        return;
      }

      // Change to the cloned repository's directory
      const repoDir = `${branchName}`;
      process.chdir(repoDir);

      // Checkout the PR branch
      exec(`git checkout ${branchName}`, (err, stdout, stderr) => {
        if (err) {
          console.error(err);
          return;
        }

        // Run prettier
        exec('prettier -w', (err, stdout, stderr) => {
          if (err) {
            console.error(err);
            return;
          }

          // Commit and push the changes
```

```

exec('git commit -a -m "Format code"', (err, stdout, stde
  if (err) {
    console.error(err);
    return;
  }

  exec(`git push origin ${branchName}`, (err, stdout, stc
    if (err) {
      console.error(err);
      return;
    }

    console.log('Code formatted and pushed.');
```

- Start the Probot app:

javascript

 Copy code

```
app.start();
```

5. Set up the bot on GitHub:

- Commit and push your code to the repository you created for your bot.
- Deploy your bot code to a server or use a cloud-based solution.
- Register your bot as a GitHub App by providing the necessary details.
- Install the bot app on the repositories where you want it to operate.

Once the bot is installed on the repositories, it will listen for new comments on PRs.

When a comment with the "/format" message is created, the